SW Engineering CSC648-848 Spring 2026

GatorBazaar

Team 07

Members	
Team Lead	Dylan Cordoni (dcordoni@sfsu.edu)
Github Maintainer	Abhay Bajaj
Back-End Lead	Sebastian Cervantez
	Ryan Zhang
Front-End Lead	Daniel De Jesus
	Kenny Camarra
	Diego Rivera-Soto

Milestone 2 Part I

History Table	
Date Submitted	April 05, 2026
Date Revised	

Table of Contents

Table of Contents.....2

1. Executive Summary:..... 3

2. Main data items and entities:.....4

3. Functional Requirements.....6

4. UI Storyboards.....7

5. High Level Architecture/Database Organization:..... 19

6. Key Risks.....22

7. Project Management.....23

8. Use of GenAI Tools:.....24

9. Team Lead Check List:.....27

1. Executive Summary:

College students often have so much stuff towards the end of their four years and even semesters. These items include textbooks from past semesters, small furniture, electronics, or school supplies. At the same time, many other students are looking for affordable alternatives instead of buying new items. However, existing marketplaces such as general online classifieds are not designed specifically for university communities and can feel inconvenient or unsafe for student use. Having to meet up in locations that aren't always safe, for example Facebook marketplace, etc. GatorBazaar is a marketplace platform designed specifically for the San Francisco State University (SFSU) community where students and professors can easily buy, sell, and exchange items within a trusted campus environment.

GatorBazaar provides a simple web-based platform where users can list items they no longer need and browse items posted by others. Listings are organized into categories such as textbooks, electronics, clothing, and dorm essentials, and many other features we have coming down the pipeline, making it easy for users to quickly find what they are looking for. Students can post items for sale, upload descriptions and prices, and connect with buyers within the SFSU community. Professors can also use the platform to share or sell academic materials or recommend resources to students. By centralizing these exchanges in one place, GatorBazaar helps reduce waste, saves students money, and creates a convenient and safe way to reuse items within the campus community.

As San Francisco State University students ourselves, our team is dedicated to making a marketplace that not only our fellow students feel confident and safe using, but one that we would use as well. With student safety and convenience at our highest priority, we believe we can make GatorBazaar the ideal cornerstone for second-hand student shopping.

2. Main data items and entities:

- **Admin Users** - Users that manage reports received by either listings or users. Able to issue warnings to users about violations or take action in the form of removing a listing or banning a registered user. Reports will be displayed on the Registered Users profile or Listing Page.
    - **Permissions -** The level of access the admin has in managing the site *(mandatory)*
- **Unregistered Users** - Users that have not have created an account on the site
    - **Access Level -** Limited to accessing the homepage, browsing, searching, and filtering listings, and viewing listing details. *(mandatory)*
- **Registered Users** - Users that have signed up on the site using a SFSU verified email
    - **Name -** The user's full name *(mandatory)*
    - **Email -** The email used to sign up for the account. Must be a verified SFSU email *(mandatory)*
    - **Role -** Defines permissions based on whether they are a buyer, seller, or admin *(mandatory)*
    - **Status -** Tracks if the user is active or banned *(mandatory)*
    - **Rating -** Rating of the user given by other users that have interacted with them *(mandatory)*

- **Listing -** An item for sale posted by and belonging to a registered user.
    - **Owner -** The registered user that the listing belongs to *(mandatory)*
    - **Title -** The name of the item for sale. *(mandatory)*
    - **Image** - An image of the item for sale set by the seller. *(mandatory)*
    - **Description -** A description of the item for sale set by the seller. *(mandatory)*
    - **Price** - Price of the item set by the user. *(mandatory)*
    - **Category** - The group of listings it belongs to *(mandatory)*
    - **Tags** - Additional searchable terms set by the seller. *(optional)*
        - **Course** - Sellers can also set an SFSU course as a tag for listings related to a course. *(optional)*
    - **Status -** Shows whether listing is currently being sold or not *(mandatory)*
    - **Date Posted** - The date that the listing was created by the seller
- **Category -** Grouping of a listing provided by the site
    - **Name -** Descriptor of the type of listing *(mandatory)*
- **Tag** - Additional searchable term used to describe listings
    - **Name -** The keyword assigned to a listing *(mandatory)*
- **Request** - Used to show a buyer's interest in a buying a listing
    - **Listing -** The listing the buyer wants to purchase *(mandatory)*

    - **Buyer -** The user sending the request *(mandatory)*
    - **Seller -** The seller of the associated listing *(mandatory)*
    - **Status -** Shows whether the request is pending, accepted, or rejected *(mandatory)*
    - **Sent Date -** The date the request was made *(mandatory)*

- **Messaging** - Allows communication between the seller and the buyer.
    - **Sender** - The user sending the message *(mandatory)*
    - **Receiver -** The user receiving the message *(mandatory)*
    - **Content -** The text sent between users *(mandatory)*
    - **Sent Time -** The time the message was sent *(mandatory)*
    - **Related Request -** The listing request associated with the conversation *(optional)*

- **Notification** - An alert that informs a user of a new message or request
    - **Recipient -** The user receiving the notification *(mandatory)*
    - **Message -** The associated message that triggers the notification *(mandatory)*
    - **Status -** Used to show if the message has been read or not *(mandatory)*
    - **Time Created -** The time the notification was created *(mandatory)*

- **Bought Listing -** A successful transaction between two users
    - **Listing -** The associated item bought *(mandatory)*
    - **Buyer -** The user who purchased the item *(mandatory)*
    - **Meet Up Location -** The recorded location where the two users exchanged goods *(mandatory)*
    - **Purchased Date -** The time the transaction occurred *(mandatory)*

- **Listing Report** - A report sent by a user against a listing that violates the sites guidelines
    - **Reported Listing -** The associated listing that is being reported *(mandatory)*
    - **Reporter -** The user sending the report *(mandatory)*
    - **Reason -** The reason for how the listing violates the sites guidelines *(mandatory)*
    - **Date -** The date the report was sent *(mandatory)*
- **User Report -** A report sent by a user against another user that has violated the sites guidelines
    - **Reported User -** The associated user that is being reported *(mandatory)*
    - **Reporter -** The user sending the report *(mandatory)*
    - **Reason -** The reason for how the listing violates the sites guidelines *(mandatory)*
    - **Date -** The date the report was sent *(mandatory)*

3. Functional Requirements

1 - must have | 2 - desired | 3 - opportunistic

Priority 1

Unregistered User

The system shall allow visitors to access the marketplace homepage.

The system shall allow visitors to browse publicly available item listings.

The system shall allow visitors to search listings using keywords.

The system shall allow visitors to view item details including title, description, price, and images.

The system shall allow visitors to filter listings by category.

Registered User

The system shall allow users to register using an SFSU email address.

The system shall allow users to receive notifications when they receive messages.

The system shall verify that registered users are associated with an SFSU email domain.

Seller

The system shall allow registered users to create listings for items they wish to sell.

The system shall allow users to upload images when creating listings.

The system shall allow users to mark listings as sold.

The system shall allow users to delete their listings.

Buyer

The system shall allow users to initiate the sending of messages to other users regarding listings.

Admin

The system shall allow administrators to remove listings that violate policies.

The system shall allow administrators to monitor marketplace activity.

Priority 2

Registered User

Seller

The system shall allow users to edit their listings.

Buyer

Admin

The system shall allow administrators to manage listing categories.

The system shall allow administrators to suspend users who repeatedly violate rules.

Priority 3

Registered User

The system shall allow users to report listings that violate platform rules.

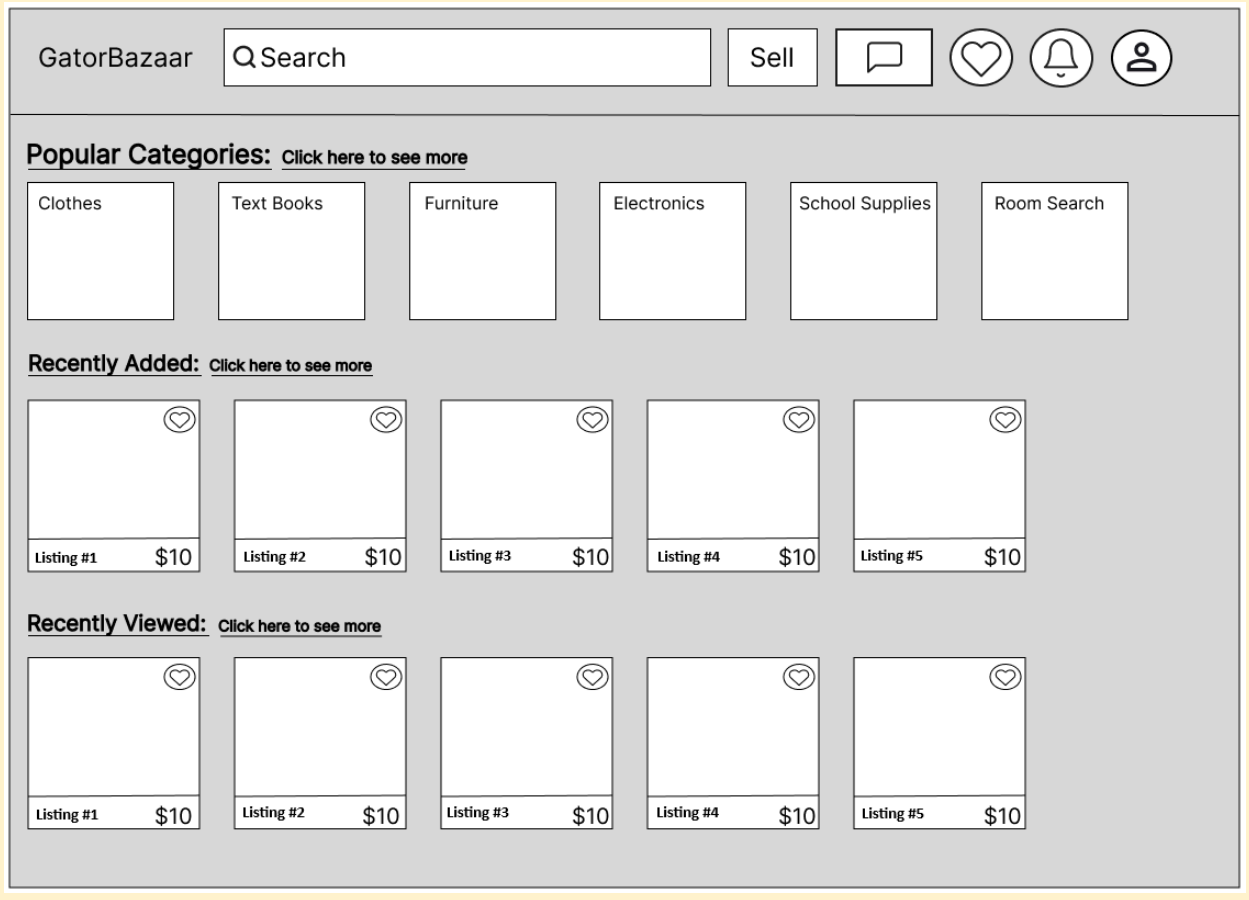
The system shall allow users to give a rating to either seller or buyer.

Admin

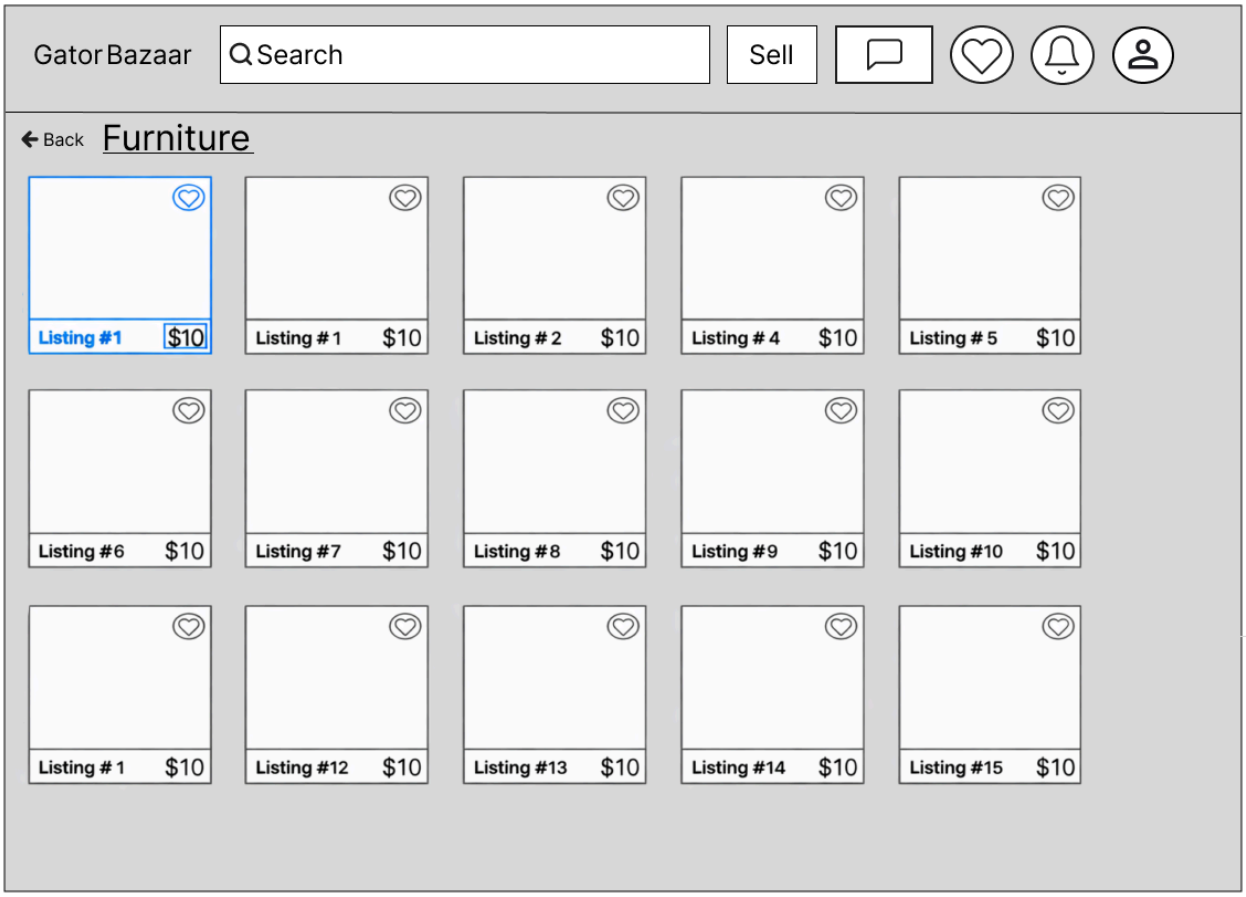
The system shall allow administrators to review reported listings.

4. UI Storyboards

Use Case 1: Buying Furniture in GatorBazaar



Landing on the home page of our application, John navigates to the furniture tab located in Popular Categories at the top of the page.

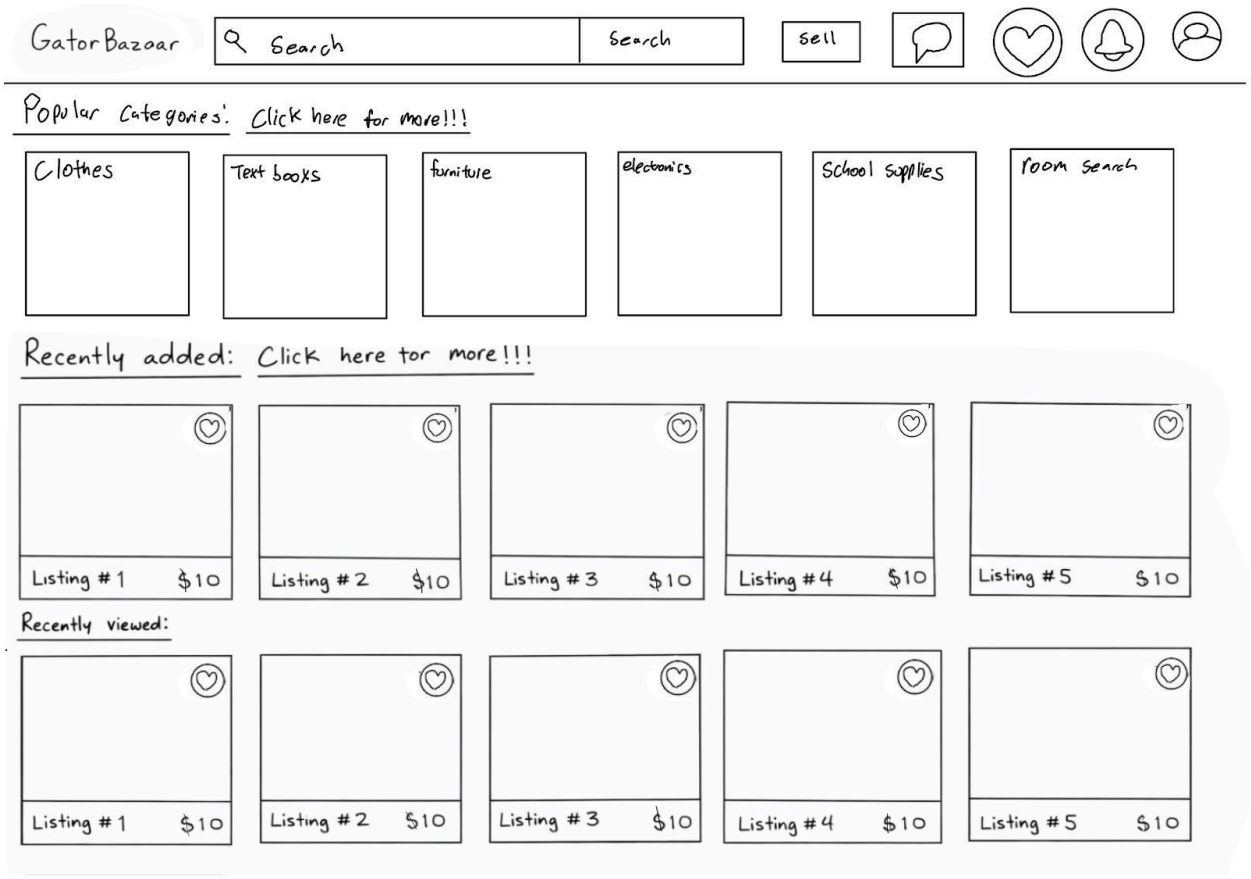


Then he can start scrolling through the page to find lists of furniture he needs.



Once he clicks on a furniture listing he likes, he will be brought to the 'buying' tab where he can find the description, and other useful images of the furniture from the seller. He can either like to save the furniture or message the seller to buy the furniture from the seller at the top right of the listing.

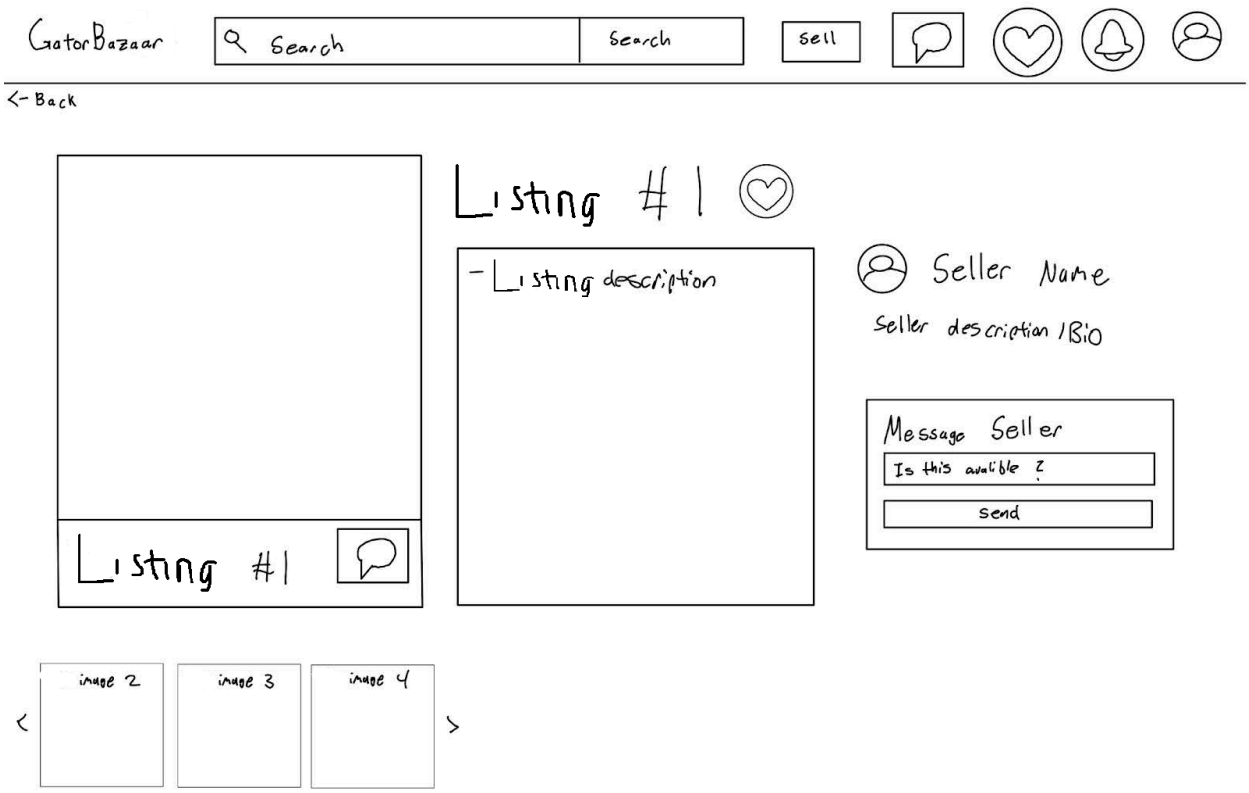
Use Case 2: Buying textbooks



Jose is looking to buy some textbooks for his classes, so he logs into GatorBazaar and searches for his Bio 101 textbook.

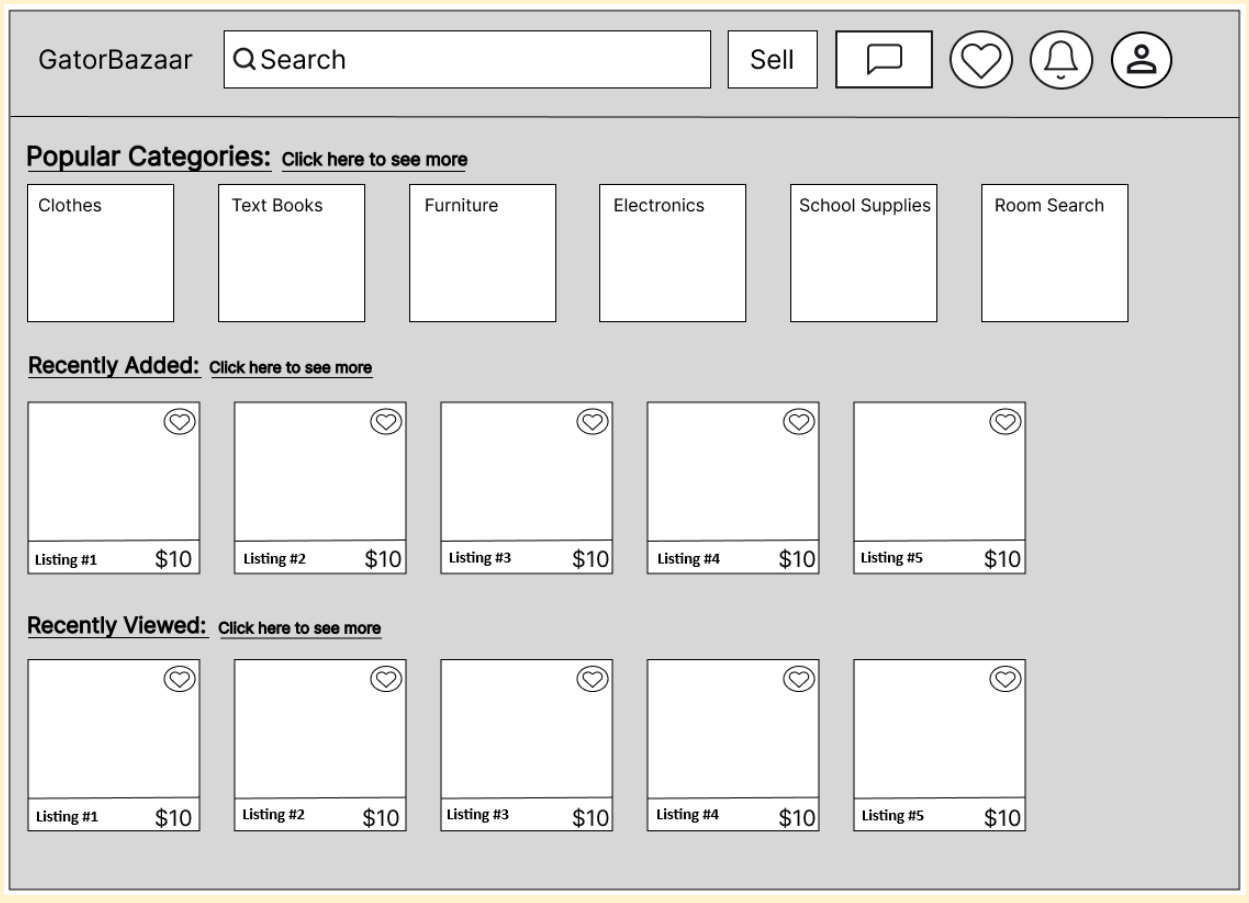


Once Jose searches for his textbook, he is prompted with the closest results to his given search. And because he added BIO 101 to his search, items with the course tag BIO 101 are displayed more often.

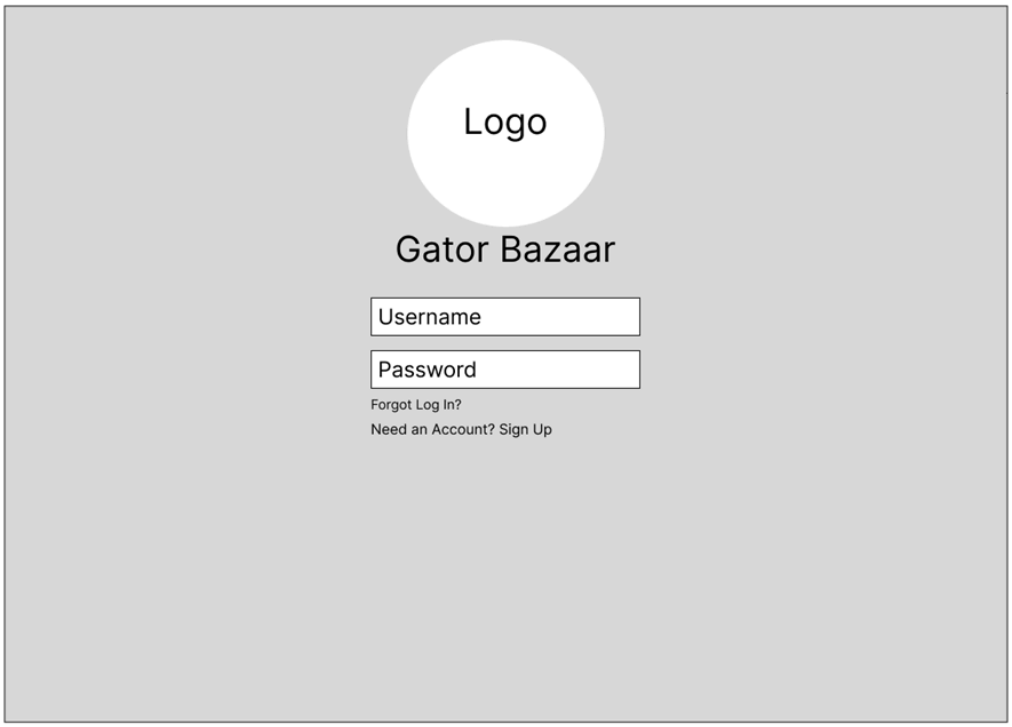


Once he selects a listing, Jose can see the seller's item description, multiple images, and the seller's bio. In order to be able to buy an item, Jose has to get in touch with the seller by sending them a message using the send message button.

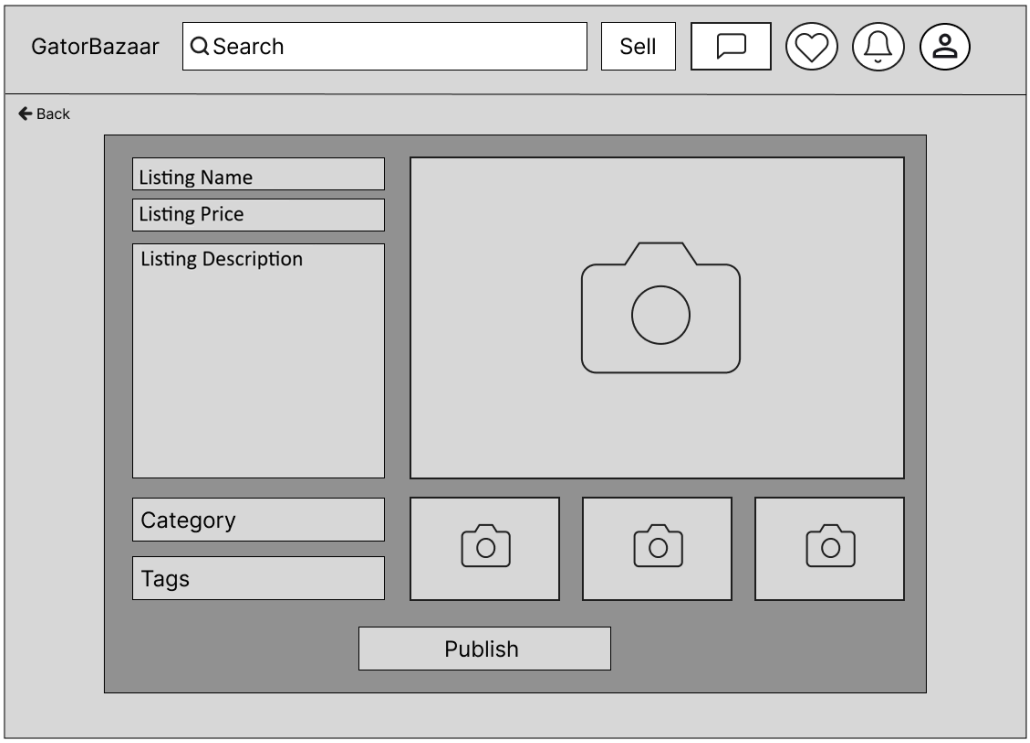
Use Case 3: Selling Items in GatorBazaar



Javier wants to sell some of the things he does not need before moving out of his apartment. However, before he can sell items in GatorBazaar, he needs an SFSU-verified account using his SFSU email, which is the user log-in located at the top right of the page.

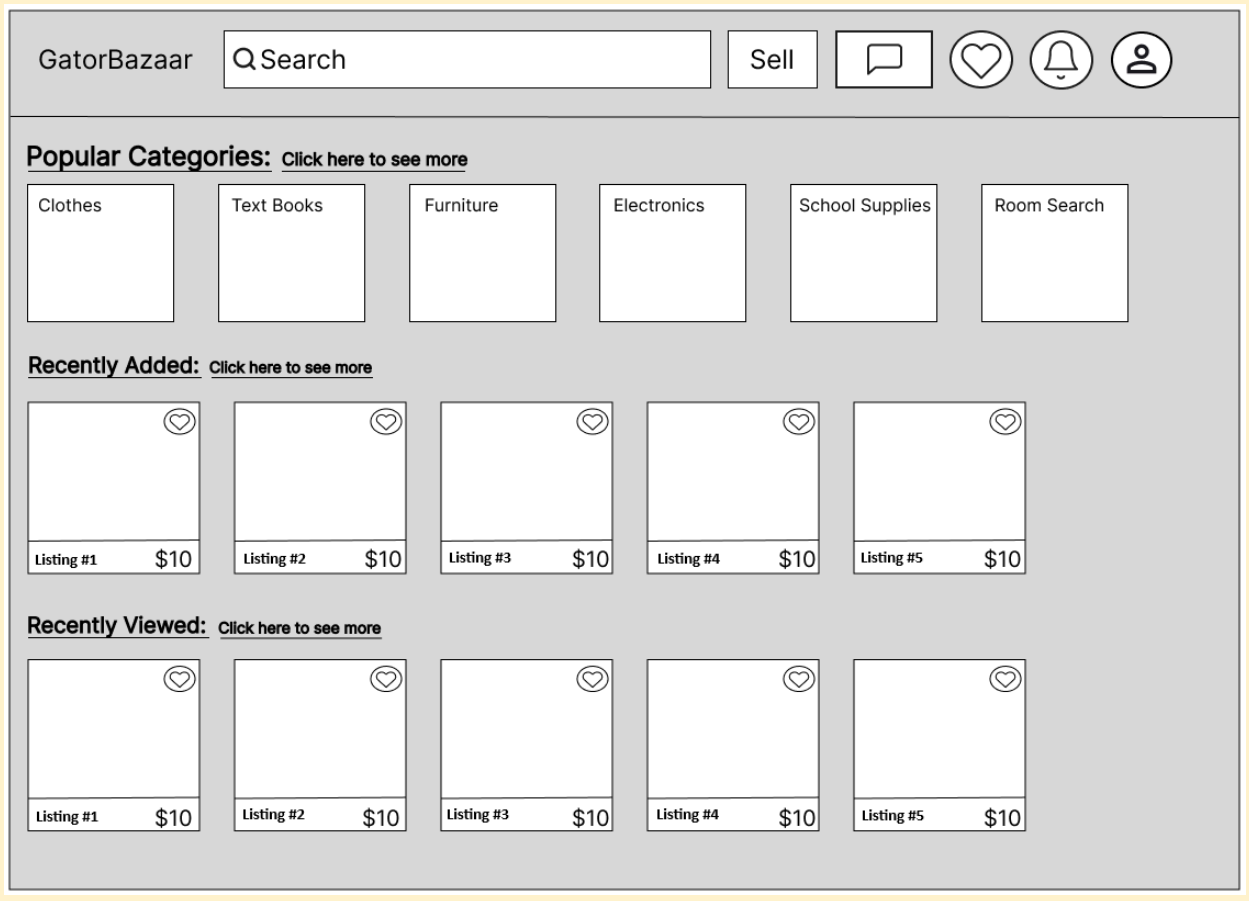


At the login page, he can sign in if he has an account set up, but he doesn't have one, so he needs to sign up with his SFSU verified email. Then he can log in with that account to start selling.

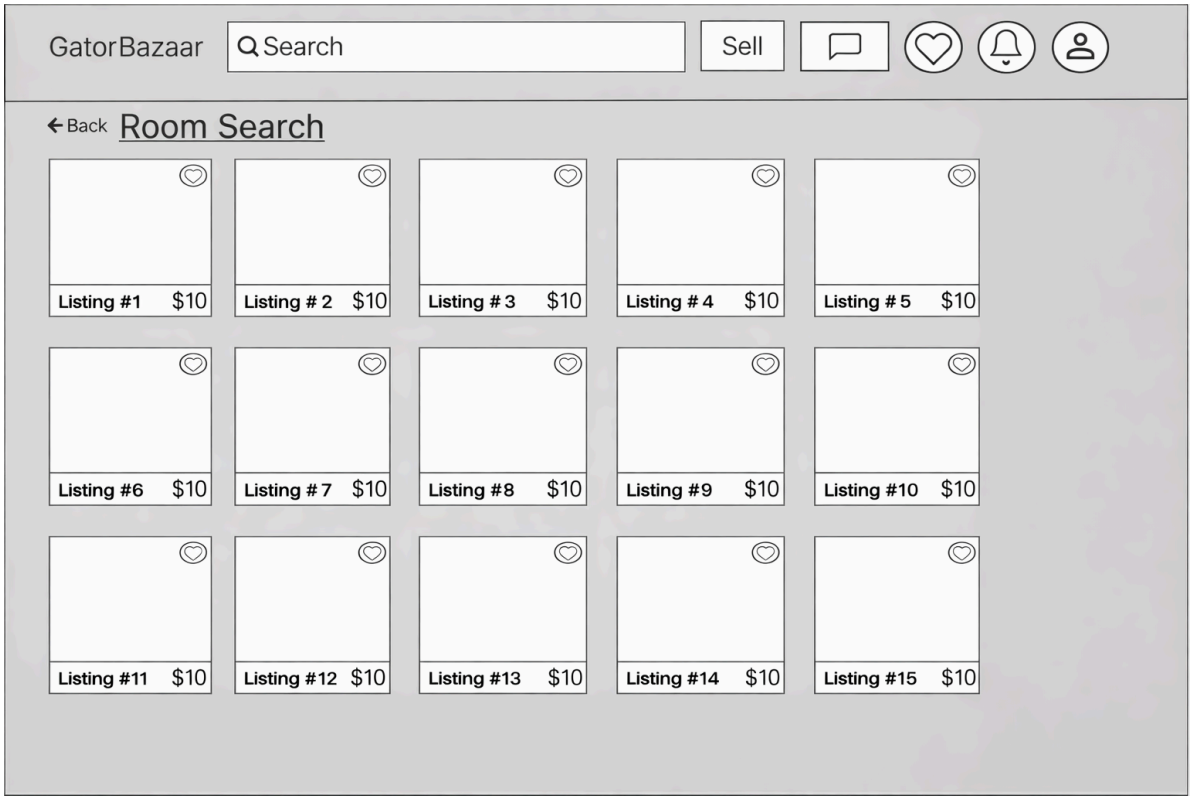


Once he signs up, he can click on the sell button at the top of the page to start selling. In the selling page, he can put the name, what type of item he's selling, the description, the location of the item, and other filters or necessary information he wants, and also upload necessary pictures of the item. When he is finished, he can press the publish button to publish the listing to GatorBazaar.

Use Case 4: Finding an Apartment in Gator Bazaar



Jose plans to move out of his dorm and live in an apartment, however he wants to cut the cost of the rent by finding a roommate. He uses Gator Bazaar to find apartment listings available in his area. At the home page, he clicks the room search category to filter out his search to apartments only.



He browses through different listings to find apartments that fit his liking. When he finds an apartment he likes he can click on it to see the listings' description.



Once he finds the apartment to his liking, he can see various information like the seller's contact, the listing description, and pictures. He can then message the seller using the message button at the top of the listing to ask more about the listing.

Use Case 5: User Admin

No mockup for the Admin UI will be displayed here as our team will be using Workbench.

5. High Level Architecture/Database Organization:

Database Organization

Architecture Pattern (MVC)

The application follows the Model-View-Controller (MVC) pattern:

- Model:
  MySQL database schema and backend data logic
- View:
  React.js frontend interface rendered in the browser
- Controller:
  Node.js / Express.js backend handling API routes, business logic, and database interaction

Search/filter architecture and implementation

Our search Algo will be SQL-based keyword search %Like. Queries will be done through node.js/expressbackend. An example will consist of **WHERE title LIKE '%keyword%'** LIKE operator. Searchable fields consist of LISTINGS: title, description, category, course - TAGS tag_name. No current plan on switching from the recommended LIKE operator.

Registered Users Tables

- user_id(Primary Key)
- Email
- Password_hash
- role(buyer-seller/admin)
- Created_at
- User_rating
- is_suspended(boolean)

Admin User Tables

- User_id (Foreign Key -> registered users)
- User_Warning (boolean)
- User_Ban (boolean)
- Remove_Listing (boolean)

Listings Table

- listing_id(Primary Key)

- User_id (Foreign Key -> registered users)
- Title
- Description
- Price
- Category
- active(boolean)
- created_at

Listing Images Table

- Image_id (primary Key)
- Listing_id (foreign Key -> Listings)
- Image_url

Tags Table

- Tag_id (Primary Key)
- Tag_name

Listing Tags Table

- Listing_id (Foreign Key-> Listing)
- tag_id (Foreign Key → Tags)

Sent Requests Table

- request_id (Primary Key)
- listing_id (Foreign Key → Listings)
- buyer_id (Foreign Key → Registered Users)
- seller_id (Foreign Key → Registered Users)
- status (pending / accepted / rejected)
- created_at

Messages Table

- message_id (Primary Key)
- request_id (Foreign Key → Sent Requests)
- sender_id (Foreign Key → Registered Users)
- receiver_id (Foreign Key → Registered Users)
- message_content
- sent_at

Notifications Table

- notification_id (Primary Key)
- user_id (Foreign Key → Registered Users)
- message_id (Foreign Key → Messages)
- is_read (boolean)
- created_at

Bought Listings Table

- purchase_id (Primary Key)
- listing_id (Foreign Key → Listings)
- buyer_id (Foreign Key → Registered Users)
- purchase_date

Listing Reports Table

- report_id (Primary Key)
- listing_id (Foreign Key → Listings)
- reported_by_user_id (Foreign Key → Registered Users)
- reason
- created_at

User Reports Table

- report_id (Primary Key)
- reported_user_id (Foreign Key → Registered Users)
- reported_by_user_id (Foreign Key → Registered Users)
- reason
- created_at

Categories Table

- category_id (Primary Key)
- category_name

Non-Trival Algorithms / Processes

By default, listings will be sorted by the time they were posted with the most recent listings appearing first. At the results page, users can further sort listings by price or the rating of the listing's seller. These attributes will be pulled from the resulting listings once the search is entered.

6. Key Risks

1. Skills Risk

Some team members may have a limited amount of experience with full-stack development in a group project. Particularly with integrating the React front-end with the Node.js/Express backend and MySQL database. This could slow development and lead to implementation issues and troubles.

Mitigation:

Team members will divide work based on strengths (front-end & back-end) and actively learn missing skills through documentation, tutorials, and help from other team members. Regular check-ins and reviews will also help to ensure that knowledge is properly shared and understood across the whole team.

2. Schedule Risk

The project includes multiple features such as listings, messaging, user ratings, and admin controls, which may be difficult to complete within the given timeline if not managed properly.

Mitigation:

Our team will prioritize core functionality first and treat additional features as secondary. Tasks will be tracked using ClickUp with clear deadlines and responsibilities to mitigate potential issues whenever possible.

3. Technical Risk

Handling image uploads for listings may introduce some challenges not originally prepared for such as storage limitations or potentially slow loading times with multiple images.

Mitigation:

Our team will use good practice techniques in order to handle image handling issues, and possibly use other storage solutions if needed. Along with image size limits that will be enforced in order to keep storage issues to a minimum.

4. Teamwork Risk

Lack of clear communication regarding feature requirements may result in inconsistent implementation across components.

Mitigation:

The team will document requirements clearly and discuss them during meetings. Shared documentation will be maintained to ensure all team members understand what is required of a feature.

5. Legal / Content Risk

Users may post inappropriate or prohibited items (copyrighted material, or otherwise prohibited content), which could create compliance or safety concerns, and open the platform up to legal action if not dealt with.

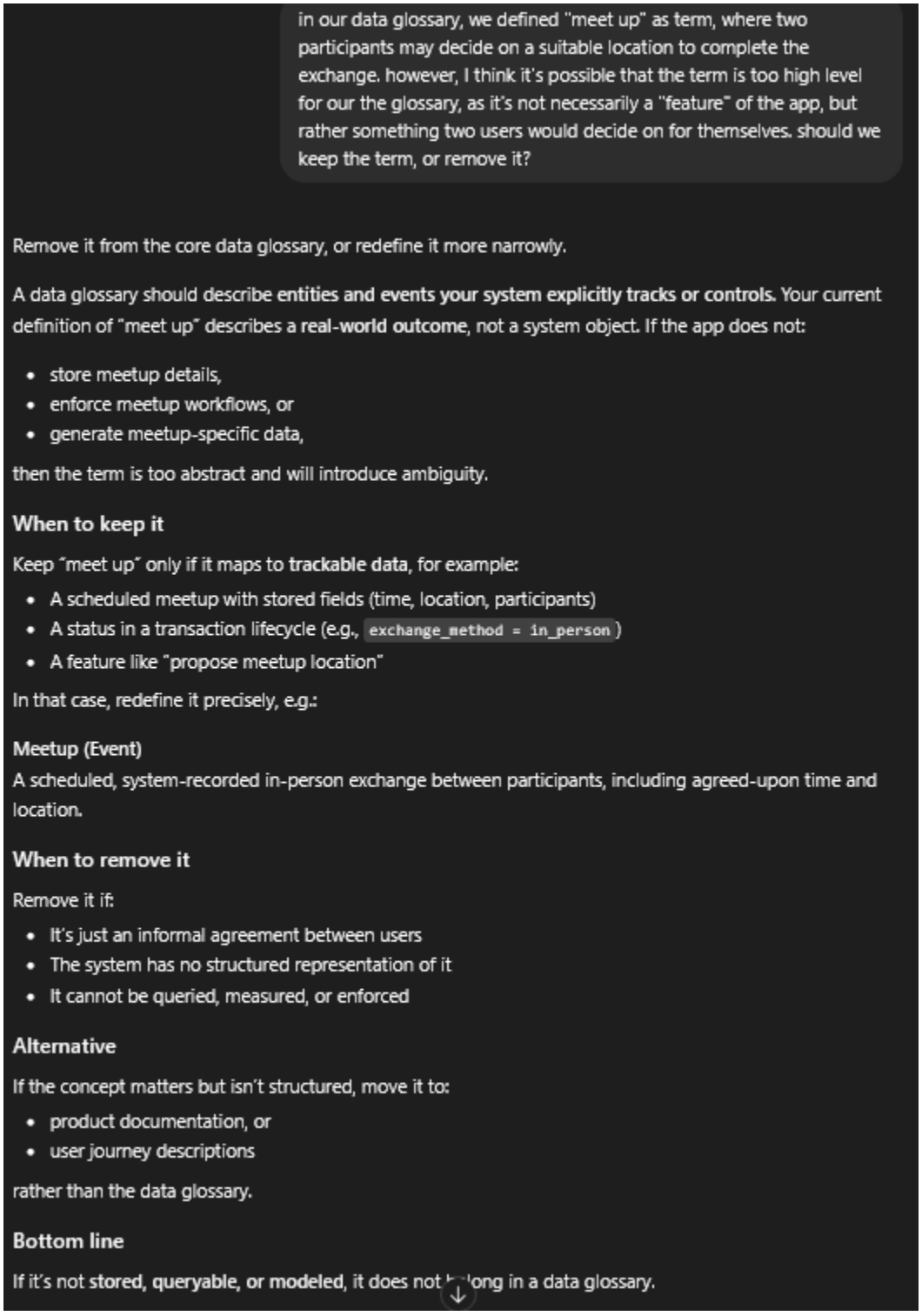
Mitigation:

The system will include reporting functionality and admin moderation tools to remove inappropriate listings. Clear platform guidelines will be established, and admins will actively monitor reported content.

7. Project Management

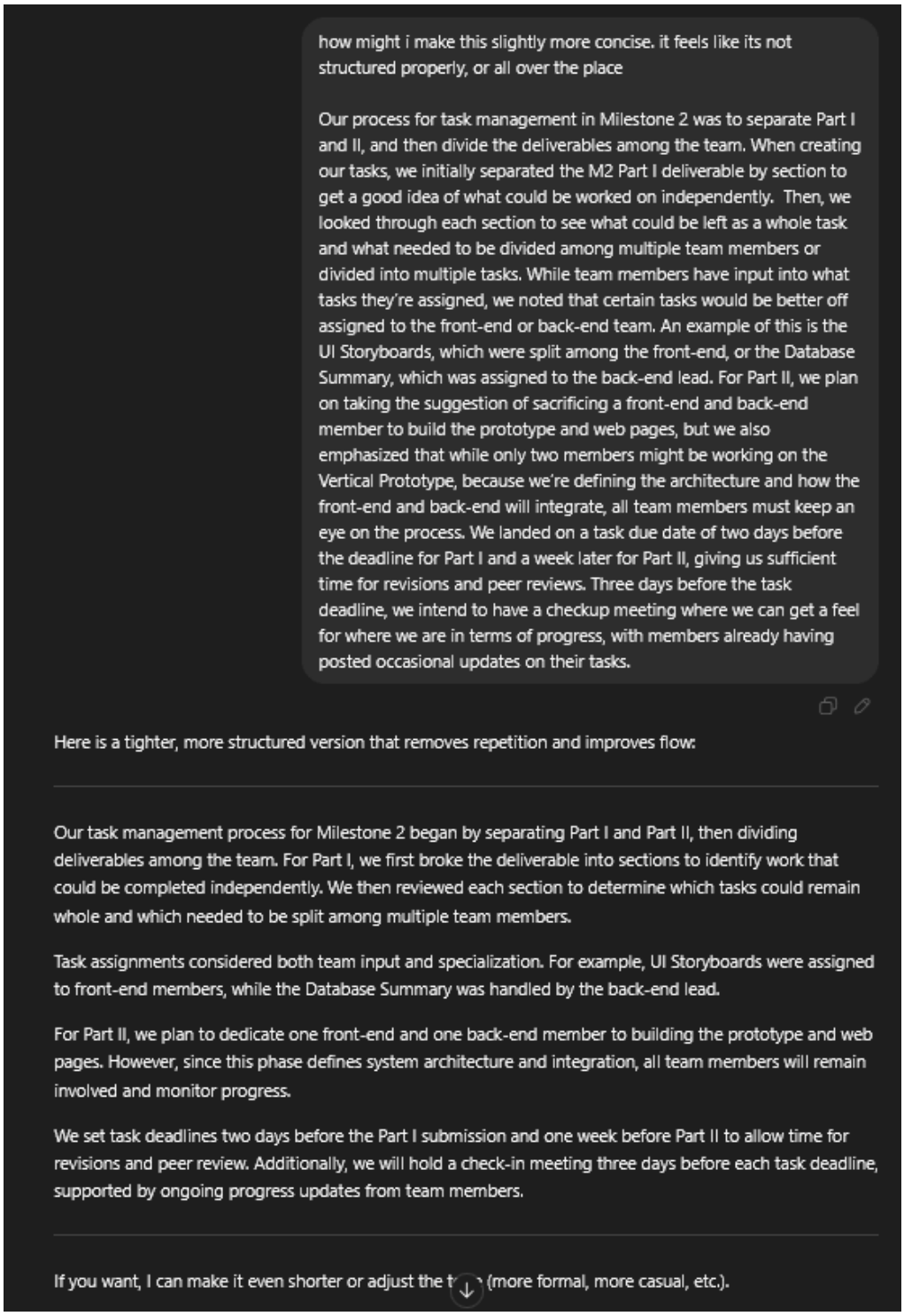
Our process for task management in Milestone 2 was to separate Part I and II, and then divide the deliverables among the team. When creating our tasks, we initially separated the M2 Part I deliverable by section to get a good idea of what could be worked on independently. Then, we looked through each section to see what could be left as a whole task and what needed to be divided among multiple team members or divided into multiple tasks. While team members have input into what tasks they're assigned, we noted that certain tasks would be better off assigned to the front-end or back-end team. An example of this is the UI Storyboards, which were split among the front-end, or the Database Summary, which was assigned to the back-end lead. For Part II, we plan on taking the suggestion of sacrificing a front-end and back-end member to build the prototype and web pages, but we also emphasized that while only two members might be working on the Vertical Prototype, because we're defining the architecture and how the front-end and back-end will integrate, all team members must keep an eye on the process. We landed on a task due date of two days before the deadline for Part I and a week later for Part II, giving us sufficient time for revisions and peer reviews. Three days before the task deadline, we intend to have a checkup meeting where we can get a feel for where we are in terms of progress, with members already having posted occasional updates on their tasks.

8. Use of GenAI Tools:



Medium Usage: We used it to get a second opinion on the necessity of the meetup term in the data glossary, as we were having second thoughts about whether it was needed, since in our

initial design it wasn't explicitly part of the website functionality but something two users would decide on for themselves. From these comments we adjusted our implementation to add the meetup location as a field that the users input and is tracked for the exchange.



Low Usage: We reworded the project management paragraph. We wanted to see if we should reword the project management paragraph, as it might be a bit too long or could be better

structured. After reviewing the output from ChatGPT, we ultimately decided that the current iteration was good enough, and that there wasn't a need to make it more concise or change things up too much. After the results, we ended up preferring the previous version because of better structurization and explanation.

9. Team Lead Check List:

Check List	
Done	So far all team members are fully engaged and attending team sessions when required
Done	Team ready and able to use the chosen back and front end frameworks and those who need to learn are working on learning and practicing
Done	Team reviewed suggested resources before drafting Milestone 2
Done	Team lead checked Milestone 2 document for quality, completeness, formatting and compliance with instructions before the submission
Done	Team lead ensured that all team members read the final Milestone 2 document and agree/understand it before submission
Done	Team shared and discussed experience with GenAI tools among themselves